

ControlPlane.ai

Risk-adaptive verification middleware for enterprise AI

ControlPlane.ai sits between an AI application and the user or tool affected by its output. It uses trusted workflow context to select the cheapest sufficient checks, then returns an auditable action: ALLOW, EDIT_REDACT, REGENERATE, BLOCK, or ESCALATE.

This repository is Team Noir's working proof of concept for Accenture Innovation Challenge 2026, Problem Track 1.

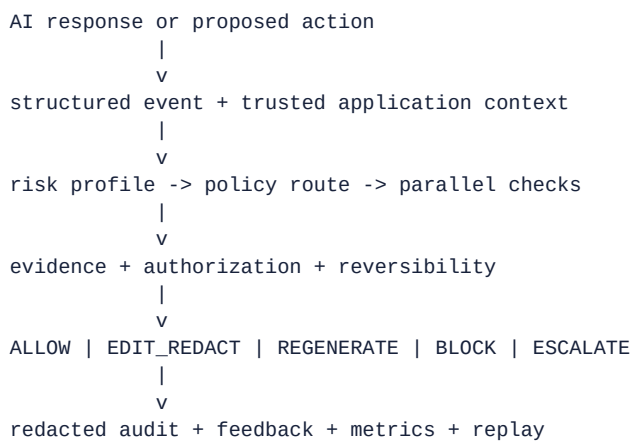
Why this prototype

A single checking pipeline creates the wrong tradeoff:

- checking every output deeply adds avoidable latency, cost, and alert fatigue;
- checking every output lightly lets high-consequence failures escape.

ControlPlane adapts verification depth to the use case, consequence, evidence, authorization, reversibility, and recent outcomes. Low-risk work can stop early; high-risk or irreversible work receives stronger checks and human review when needed.

Core flow



The Streamlit console makes the tradeoff visible for every scenario: AI input and candidate output, final action, risk tier, measured latency, model calls, stop reason, checker outcomes, evidence trace, and raw JSON.

Stage 2 coverage

Stage 2 area	Implemented mechanism
Detection	Engineering safety rules, PII/secret checks, claim extraction, governed retrieval, entitlement checks, history, and optional Groq AI-as-judge
Decision logic	Four risk tiers, four evidence states, separate authorization, hard vetoes, and five proportional actions
Architecture	Inline FastAPI middleware, tier-local parallel checks, early stopping, and policy latency budgets
Governance	Versioned YAML policies, governed source authority/status, checksums, redacted SQLite audit, and drift-aware replay
Feedback	Reviewer labels update adverse-outcome history without changing the original audit record
Metrics	Decisions, risk, stop reasons, latency, checks, estimated cost units, model calls, and feedback summaries

The prototype covers two simulated enterprise workflows: an engineering agent and a customer-support assistant. It uses illustrative data; no proprietary enterprise data or production integration is required.

What the decisions mean

Action	Meaning
ALLOW	Required checks passed; the candidate may continue.
EDIT_REDACT	Localized sensitive data was removed before release.
REGENERATE	A LOW/MEDIUM response is contradicted, unsupported, or uncertain; return it for correction.
BLOCK	A critical policy, authorization, secret, or reversibility rule prevents execution.
ESCALATE	HIGH/CRITICAL uncertainty, missing approval, or an unresolved mandatory check requires a human.

POST /evaluate is a one-shot evaluation; ControlPlane does not call the source AI again, so it cannot create an internal regeneration loop. A host integration should cap regeneration at one or two attempts, include the failure reason in the retry, reject identical output, and escalate or return a safe fallback when the retry or latency budget is exhausted. The PoC also raises repeated adverse outcomes to higher risk through its historical signal.

Demo cases

Scenario	What it demonstrates	Expected action
Safe development edit	LOW risk, deterministic route, early stop	ALLOW
Unbounded production delete	Critical failure is not averaged away	BLOCK
PII plus false refund claim	Privacy editing cannot hide an evidence failure	REGENERATE
Informational answer without evidence	Cheap LOW-risk correction without an LLM call	REGENERATE
Judge-assisted refund correction	MEDIUM risk uses Groq but avoids unnecessary human review	REGENERATE
High-risk financial guarantee	Unsupported financial commitment reaches human review	ESCALATE
Unauthorized cancellation	Evidence cannot substitute for identity or approval	BLOCK

The three live judge fixtures deliberately produce different proportional outcomes: the MEDIUM informational case regenerates automatically, while HIGH and CRITICAL unsupported commitments escalate.

Run locally

Requirements: Git and Python 3.11.

Windows PowerShell

```
git clone https://github.com/hrushikeshroop/Accenture_Hackathon.git
cd Accenture_Hackathon
py -3.11 -m venv .env
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
python scripts\run_demo.py --fresh-db
```

macOS or Linux

```
git clone https://github.com/hrushikeshroop/Accenture_Hackathon.git
cd Accenture_Hackathon
python3.11 -m venv .env
source .venv/bin/activate
python -m pip install -r requirements.txt
cp .env.example .env
python scripts/run_demo.py --fresh-db
```

Open the Streamlit URL printed in the terminal. API documentation is available at <http://127.0.0.1:8000/docs>. The launcher binds both services to localhost and stops them together. --fresh-db prevents earlier demo history from changing the walkthrough.

Optional live Groq judge

The Groq endpoint and free-tier model are fixed in code. Add only your own key to the ignored .env file:

```
GROQ_API_KEY=your_new_key
```

Then run:

```
python scripts\run_live_judge_demo.py
```

The judge receives only the redacted candidate and accumulated retrieval trace. It cannot replace deterministic critical vetoes, grant authorization, or use its own knowledge as evidence. Never commit .env; revoke any key exposed in chat, source, screenshots, or recordings.

Verification

```
pytest -q
python scripts\run_all_scenarios.py
python evaluation\run_evaluation.py
python scripts\run_policy_replay_demo.py
python scripts\run_model_judge_demo.py
```

Verified results on the included PoC fixtures:

- 99 automated tests passed; 3 live Groq tests are opt-in and skipped by default.
- 17 of 17 labelled scenarios matched their expected actions.
- False-block rate: 0.0 on the included labelled fixtures.
- Unsafe-escape rate: 0.0 on the included labelled fixtures.
- Average checks executed: 4.53 per scenario.
- Deterministic evaluation made zero external model calls.

To run the opt-in live integration tests in PowerShell:

```
$env:CONTROLPLANE_RUN_LIVE_JUDGE="1"
pytest -m live -q
Remove-Item Env:CONTROLPLANE_RUN_LIVE_JUDGE
```

These figures describe only the included simulated fixtures; they are not claims of production accuracy, safety, throughput, or provider availability.

Repository map

controlplane/	Middleware contracts, routing, checks, decisions, API, and audit
dashboard/	Streamlit demonstration console
policies/	Versioned risk and verification policies
knowledge/	Illustrative governed evidence sources
scenarios/	Labelled engineering and support events
evaluation/	Repeatable fixture evaluation and baseline
scripts/	Launch, scenario, replay, judge, and release utilities
tests/	Unit, regression, integration, UI, and opt-in live checks

Scope

- Platform integrations and enterprise data are simulated.
- Retrieval is governed structured fact lookup, not semantic document search.
- Rule-based command, claim, and PII detection is intentionally bounded.
- SQLite and Streamlit are local PoC components, not production architecture.
- Authentication, multi-tenant isolation, load testing, regulatory certification,

comprehensive bias/prompt-injection coverage, and production monitoring are outside this prototype.

Team

Team Noir, IIT Kanpur - M. Enoch Emmanuel, Jayanth Matam, and Hrushikesh Roop Avvari.